

Automatización del Análisis de Puntos de Función mediante Inteligencia Artificial: Implementación de un Web Service

Britos, Sofia Ailen

Laboratorio de Investigación de Software

Departamento de Ingeniería en Sistemas de Información

Facultad Regional Córdoba - Universidad Tecnológica Nacional

Resumen

Este proyecto presenta una propuesta de automatización del Análisis de Puntos de Función (FPA) según el estándar NESMA, mediante Inteligencia Artificial, implementada como un Web Service.

El sistema recibe un documento PDF, del cual extrae automáticamente las user stories escritas en lenguaje natural, y emplea un modelo preentrenado de procesamiento de lenguaje para clasificarlas en las categorías de FPA (Entradas Externas, Salidas Externas, Consultas Externas, Archivos Lógicos Internos y Archivos de Interfaz Externa). Una vez realizada la clasificación, se procede al cálculo de la estimación de puntos de función. La arquitectura propuesta está desarrollada en Spring Boot e integra servicios de IA accesibles vía API.

Palabras claves: Análisis de Puntos de Función (FPA), NESMA, Inteligencia Artificial, Web Service, Users Stories, Entradas Externas (EI), Salidas Externas (EO), Consultas Externas (EQ), Archivos Lógicos Internos (ILF), Archivos de Interfaz Externa (EIF), Categorías de FPA, API.

Introducción

La estimación de software es un proceso clave en la ingeniería de sistemas, ya que permite dimensionar el esfuerzo, costo y tiempo necesarios para el desarrollo de proyectos. Dentro de las metodologías existentes, el FPA se ha consolidado como un estándar ampliamente utilizado, en

particular bajo las directrices de la NESMA[1] (Netherlands Software Metrics Association).

El FPA permite medir el tamaño funcional de un sistema a partir de los requerimientos de usuario, clasificando sus funciones en categorías.

A pesar de sus ventajas, la aplicación del FPA de manera tradicional presenta limitaciones: requiere analistas con experiencia, demanda tiempo significativo y puede verse afectada por la subjetividad en la interpretación de los requerimientos. En un contexto donde los equipos de desarrollo adoptan metodologías ágiles y ciclos de entrega cada vez más cortos, la necesidad de herramientas que aceleren y estandaricen este proceso resulta evidente.

Este proyecto constituye una **segunda versión** [2] de una propuesta de automatización del FPA.

En la **primera** versión, el enfoque estuvo orientado a **proyectos tradicionales** basados en metodologías como PUD/RUP (Proceso Unificado de Desarrollo), donde se implementaron de manera automatizada los métodos de conteo IFPUG y NESMA mediante algoritmos específicos. Para ello se toman como **fuentes de información**

los artefactos característicos de dichos procesos, tales como diagramas de casos de uso, modelos de objetos y clases de análisis generados con herramientas UML o CASE.

La **segunda** versión, presentada en este trabajo, avanza un paso más al incorporar técnicas de Inteligencia Artificial para el análisis de proyectos en **entornos ágiles**, donde los requerimientos se expresan en forma de user stories redactadas en lenguaje natural. La integración de modelos de lenguaje permite procesar automáticamente estas descripciones, identificar funciones de datos y transaccionales, asignarles complejidad y estimar el esfuerzo en puntos de función, ofreciendo un enfoque más flexible y adaptado a la naturaleza iterativa e incremental de los proyectos ágiles.

En los últimos años, la Inteligencia Artificial (IA) y, en particular, el Procesamiento de Lenguaje Natural (PLN), han demostrado un gran potencial para asistir en tareas de análisis de texto, clasificación y extracción de información. Estas capacidades abren la posibilidad de aplicar modelos preentrenados al análisis de requerimientos en lenguaje natural, con el fin de automatizar parte del proceso de estimación funcional.

El objetivo principal de este trabajo es demostrar la viabilidad técnica de integrar modelos de IA en el proceso de análisis FPA, evaluando sus beneficios en términos de reducción de tiempo, estandarización del proceso y disminución del sesgo humano.

Para comprender adecuadamente esta propuesta, resulta necesario introducir el marco metodológico que la sustenta, en

particular el estándar definido por la **NESMA**, ampliamente utilizado en la estimación funcional de proyectos de software.

1. Fundamentación teórica

1.1 NESMA

Significa **Netherlands Software Metrics Association**. Es una organización que desarrolló **estándares para medir el tamaño funcional de software** usando **Análisis de Puntos de Función (FPA)**.

En otras palabras: NESMA **define reglas y guías precisas** para calcular los puntos de función de un sistema, asegurando que distintos analistas puedan estimar el tamaño funcional de manera **consistente y comparable**.

Algunas características clave de NESMA:

- Es **independiente de la tecnología**: no importa el lenguaje de programación ni la plataforma.
- Define claramente las **categorías de funciones**: EI, EO, EQ, ILF y EIF.
- Establece reglas para **asignar complejidad** a cada función y, por tanto, calcular los puntos de función totales.
- Facilita la **comparación entre proyectos** y la integración con estimaciones de costo y tiempo.

1.2 Análisis de Puntos de Función según NESMA

El **Análisis de Puntos de Función (FPA, Function Point Analysis)** es una técnica estandarizada para medir el tamaño funcional de un sistema de software a

partir de sus requerimientos, independiente del lenguaje de programación o la tecnología utilizada. Su objetivo es cuantificar la funcionalidad entregada al usuario, lo que permite estimar de manera más objetiva el esfuerzo, tiempo y costo necesarios para desarrollar o mantener un sistema.

Existen varios estándares de FPA, siendo uno de los más utilizados el de **NESMA**, que proporciona directrices precisas para la identificación, clasificación y valoración de las funciones elementales de un sistema. Según NESMA, las funciones se clasifican en cinco categorías principales:

1. **Entradas Externas (EI – External Input):** Transacciones que permiten al usuario ingresar información al sistema, generando un cambio en los datos internos.
2. **Salidas Externas (EO – External Output):** Transacciones que producen información derivada de datos internos, entregada al usuario, generalmente mediante reportes o mensajes.
3. **Consultas Externas (EQ – External Query):** Solicitudes de información que no modifican los datos internos, pero permiten obtener respuestas específicas del sistema.
4. **Archivos Lógicos Internos (ILF – Internal Logical File):** Conjuntos de datos controlados y mantenidos dentro del sistema, que representan entidades del negocio o del dominio.
5. **Archivos de Interfaz Externa (EIF – External Interface File):** Conjuntos de datos usados por el sistema pero mantenidos por otro

sistema externo.

Una vez identificadas y clasificadas las funciones, se asigna a cada una un **nivel de complejidad** (bajo, medio o alto), basado en criterios como el número de archivos referenciados y el número de campos de datos. Cada categoría y nivel de complejidad tiene un **valor en puntos de función**, que permite calcular el tamaño total del sistema mediante la suma ponderada de todas las funciones.

El enfoque de NESMA destaca por su **claridad y estandarización**, facilitando la comparación entre proyectos y la integración con metodologías de estimación y planificación de software. Además, proporciona reglas explícitas para manejar transacciones compuestas, funciones repetidas y variabilidad en la interpretación de los requerimientos, reduciendo el sesgo subjetivo del analista.

1.3 Métodos de análisis de puntos de función

Nesma reconoce tres métodos de análisis de puntos de función:

- Análisis detallado de puntos de función
- Análisis de puntos de función estimados.
- Análisis de puntos de función indicativos.

Cada uno de estos métodos, constituye un método autónomo de medición de tamaño funcional por sí solo.

El método FPA de alto nivel o estimado y el método FPA indicativo no requieren requisitos detallados del usuario, mientras

que el tamaño funcional determinado utilizando estos métodos es muy cercano al tamaño funcional determinado utilizando el método FPA detallado. Es por eso que estos dos métodos son muy adecuados para aplicarse temprano en el ciclo de vida del desarrollo de software o en caso de que sea necesario determinar rápidamente el tamaño funcional.

a. FPA detallado

Este es el método habitual de análisis de puntos de función y se realiza de la siguiente manera:

- Determinar todas las funciones de todos los tipos de funciones (ILF, EIF, EI, EO, EQ)
- Califique la complejidad de cada función (baja, media, alta)
- Calcular el recuento total de puntos de función no ajustados

b. FPA de alto nivel (también conocido como FPA estimado)

El método de análisis de puntos de función de alto nivel se realiza de la siguiente manera:

- Determinar todas las funciones de todos los tipos de funciones (ILF, EIF, EI, EO, EQ)
- Califique la complejidad de cada función de datos (ILF, EIF) como Baja y de cada función transaccional (EI, EO, EQ) como Promedio
- Calcular el recuento total de puntos de función no ajustados

Entonces, la única diferencia de esta aproximación con el método de análisis

detallado de puntos de función es que la complejidad no se determina por función individual, sino de forma predeterminada.

c. FPA indicativo

El método de análisis de puntos de función indicativos se realiza de la siguiente manera:

- Determinar el número de funciones de datos (ILF y EIF);
- Calcule el recuento total de puntos de función no ajustados de la aplicación de la siguiente manera:

tamaño indicativo (fp) = 35 x número de ILF + 15 x número de EIF

Por tanto, esta aproximación se basa únicamente en los archivos lógicos (ILF y EIF).

El análisis de puntos de función indicativos se basa en el supuesto de que habrá alrededor de tres EI (para agregar, cambiar y eliminar información en el ILF), dos EO y un EQ en promedio para cada ILF, y alrededor de un EO y un EQ por cada EIF.

1.4 Asignación de puntos de función según complejidad - Nesma

Tras la clasificación, se asigna una **complejidad** a cada función (baja, media o alta), y se calculan los **puntos de función** de acuerdo con la siguiente tabla de Nesma:

Tipo de función	Bajo	Promedio	Alto
Archivo lógico interno (ILF)	7	10	15

Archivo de interfaz externa (EIF)	5	7	10
Entrada externa (EI)	3	4	6
Salida externa (EO)	4	5	7
Consulta externa (EQ)	3	4	6

2. Propuesta de solución

En un principio, se observó que la mayoría de los equipos de desarrollo adoptan metodologías ágiles, donde los requerimientos se documentan principalmente mediante user stories. Para automatizar el FPA, se decidió que el sistema debía recibir estas user stories de los proyectos y procesarlas de manera automática.

El flujo de análisis planteado consiste en:

1. **Extracción de entidades y casos de uso relacionados:** A partir de las user stories, se identifican las entidades funcionales y los use case asociados, que son relevantes para la medición de FPA.
2. **Clasificación según FPA de NESMA:** Cada entidad o caso de uso se categoriza en una de las cinco funciones elementales (EO, EI, EQ, ILF y EIF).
3. **Asignación de complejidad:** Una vez categorizadas, se asigna una complejidad por defecto a cada función, permitiendo el cálculo de

los puntos de función de manera automática.

Durante la fase de diseño se evaluaron tres enfoques de implementación:

1. **Algoritmo basado en lógica pura:** Desarrollo de reglas y patrones heurísticos para identificar y clasificar entidades y casos de uso a partir del texto de las user stories.
2. **Integración de un modelo preentrenado:** Uso de un modelo de procesamiento de lenguaje natural ya entrenado, con comunicación vía API, para realizar la clasificación y extracción de información de manera más flexible y robusta frente a la variabilidad del lenguaje natural.
3. **Entrenamiento de un modelo específico:** Creación de un modelo entrenado o ajustado (*fine-tuned*¹) con un conjunto de user stories previamente etiquetadas, lo que permite adaptar la IA al dominio particular del proyecto, mejorar la precisión en la clasificación y capturar patrones específicos de redacción de los equipos de desarrollo.

Finalmente, se optó por la **integración del modelo preentrenado**, priorizando su capacidad de manejar distintos estilos de redacción, la reducción de mantenimiento de reglas específicas y la escalabilidad frente a proyectos de mayor tamaño. Sin

¹ Un modelo *fine-tuned* es un modelo previamente entrenado (*pretrained*) sobre un gran conjunto de datos generales, al que luego se le realiza un **ajuste fino** con un conjunto de datos más específico y reducido, adaptándolo a una tarea concreta.

embargo, el entrenamiento de un modelo propio se plantea como un paso futuro para mejorar la precisión y personalización del análisis.

2.1 Alcance dentro del marco ágil

En el contexto de las metodologías ágiles, donde los requerimientos se documentan mediante *user stories* que evolucionan de manera iterativa, la aplicación del análisis automatizado de Puntos de Función puede abordarse desde dos niveles de alcance.

En primer lugar, el **uso principal propuesto es durante la fase inicial del proyecto**, cuando ya se dispone de un *backlog completo o suficientemente representativo*. En esta etapa, el análisis automatizado permite obtener una estimación base del tamaño funcional del sistema, facilitando la planificación global, la comparación entre proyectos y la proyección de recursos. Este enfoque aprovecha el potencial del modelo para procesar grandes volúmenes de *user stories* y producir una medición objetiva y reproducible, reduciendo la variabilidad inherente a las estimaciones manuales.

De manera complementaria, el proceso también puede aplicarse a nivel de Sprint. En este caso, el análisis se focaliza en las *user stories* nuevas o modificadas, permitiendo mantener la trazabilidad de los cambios en el esfuerzo estimado y reflejar la evolución funcional del sistema a lo largo de las iteraciones. Aunque este uso iterativo conlleva un mayor costo de procesamiento respecto a técnicas colaborativas como *planning poker*, resulta útil en contextos donde se requiere auditoría, control de métricas o seguimiento continuo del crecimiento funcional del producto.

En conjunto, esta estrategia posiciona al análisis FPA automatizado como una herramienta **complementaria** dentro del marco ágil: útil para la planificación inicial y valiosa para el seguimiento de cambios, sin sustituir las dinámicas colaborativas propias del desarrollo ágil.

3. Consideraciones sobre el modelo

3.1 Elección del modelo

Dentro de las opciones gratuitas disponibles en Hugging Face, se seleccionó el modelo **mistralai/Mistral-7B-Instruct-v0.2** por su equilibrio entre capacidad de comprensión de instrucciones (instruct-tuned) y eficiencia.

A continuación, se presentan los **requisitos técnicos** del modelo y su **justificación de elección**.

3.1.1 Requisitos técnicos del modelo

Según las especificaciones automatizadas de Hugging Face, para realizar tareas de inferencia con **Mistral-7B-Instruct-v0.2**² se necesita aproximadamente:

- **16-bit (float16/bfloat16):**
 - **Tamaño total:** ~13.74 GB de VRAM
 - **Capa más grande:** ~432 MB
- **8-bit (int8):**
 - **Tamaño total:** ~6.87 GB de VRAM
- **4-bit (int4):**

²<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2/discussions/20>

- **Tamaño total:** ~3.44 GB de VRAM

Además, una fuente adicional indica que el modelo original tiene unos **7.3 mil millones de parámetros** y requiere en torno a **16 GB de GPU VRAM** para su funcionamiento estándar.³

Para integrarse con facilidad en diferentes entornos, el modelo también permite técnicas de **cuantización**, especialmente en 4 bits (load_in_4bit) utilizando librerías como **BitsAndBytes**, lo que reduce drásticamente los requerimientos de memoria sin sacrificar significativamente la calidad.⁴

3.2 Justificación de la elección

- **Gratis y accesible:** Al estar disponible en Hugging Face y no requerir suscripciones, se ajusta a los recursos disponibles del proyecto.
- **Capacidad instructiva:** Diseñado como modelo instruct-tuned, es adecuado para tareas de clasificación con prompts estructurados, como las de NESMA.
- **Balance entre potencia y recursos:** El soporte para cuantización (hasta 4-bit) permite correr el modelo incluso en hardware limitado, ideal para plataformas como Google Colab.
- **Largo contexto:** Soporta secuencias largas (hasta ~32K tokens), lo cual resulta útil cuando

se procesan múltiples user stories en un mismo prompt.⁵

3.3 Alternativas de modelos

Este modelo representa un punto de partida sólido, pero la investigación continuará con otras alternativas:

- **google/gemini-3-270m:** modelo más pequeño y posiblemente más eficiente para hardware limitado.
- **Modelos DeepSeek:** por explorar mejoras en precisión o adaptabilidad al dominio específico.
- **Entrenamiento fino (fine-tuning):** ajustar el modelo a las particularidades de proyectos ágiles mejorando su rendimiento y exactitud.

4. Implementación

El sistema se implementa como un **Web Service** desarrollado en **Spring Boot**, siguiendo una **arquitectura monolítica** y ofreciendo una **API REST** para la interacción con los usuarios y otros sistemas. La API cuenta con tres endpoints principales:

1. **Procesar PDF:** recibe un documento PDF y extrae automáticamente todas las user stories que contiene.
2. **Recibir user story individual:** permite enviar una user story de manera individual para su análisis.
3. **Recibir múltiples user stories:** admite el envío de varias user stories en un solo request,

³ <https://docs.mistral.ai/getting-started/models/weights/>

⁴ https://github.com/huggingface/transformers/blob/main/docs/source/en/model_doc/mistral.md

⁵ <https://apxml.com/models/mistral-7b-instruct-v0-2>

optimizando el procesamiento de proyectos con mayor número de requerimientos.

Una vez recibidas las user stories, estas se combinan con un **prompt** previamente diseñado y se envían al modelo de IA preentrenado. El rol del modelo es clasificar cada historia en una de las categorías definidas por NESMA (EI, EO, EQ, ILF y EIF) y, adicionalmente, asignarle un nivel de complejidad (baja, media o alta). El resultado se devuelve en formato **JSON**, donde cada elemento contiene la descripción de la función, el tipo identificado y la complejidad estimada.

4.1 Integración con el modelo de IA

Dado que el modelo seleccionado se ejecuta en un entorno externo, se diseñó un flujo de comunicación entre el Web Service principal y un servicio auxiliar responsable de interactuar con el modelo de lenguaje.

Para este fin, se utilizó **Google Colab**[3] como entorno de ejecución, debido a su disponibilidad de recursos de cómputo y facilidad para experimentar con modelos.

En Colab se implementó una **API ligera** mediante **FastAPI**⁶, la cual expone un único endpoint (**/generate**) encargado de recibir las user stories, ejecutar el modelo y retornar la clasificación correspondiente.

Para permitir la comunicación entre el Web Service desarrollado en Spring Boot y la API alojada temporalmente en Colab, se

empleó **Ngrok**⁷, herramienta que genera una URL pública y segura que actúa como túnel hacia el servidor local de FastAPI.

De este modo, el servicio en Spring Boot puede enviar solicitudes HTTP hacia la API en Colab como si se tratara de un servicio externo en la nube.

4.2 Procesamiento de la respuesta y limpieza de datos

El modelo devuelve su salida en formato texto, con variaciones sintácticas según el contexto o el estilo de redacción del modelo.

Por este motivo, se desarrolló una función de limpieza y normalización de la respuesta, cuyo objetivo es garantizar que el texto recibido contenga un JSON válido y estructurado antes de su análisis.

El proceso de limpieza realiza las siguientes tareas:

1. **Identifica** el bloque que contiene el objeto JSON.
2. **Elimina** posibles caracteres adicionales o comentarios generados por el modelo.
3. **Valida** la estructura y transforma el texto en un objeto JSON interpretable por Java.

Una vez validado, el Web Service recorre el contenido de la respuesta y, para cada función clasificada, asigna los valores correspondientes según la **tabla de complejidad definida** (detallado antes en el punto 1.4).

A partir de estos datos, el sistema genera automáticamente:

⁶ <https://fastapi.tiangolo.com/>

⁷ <https://sendbird.com/developer/tutorials/what-is-ngrok>

- un **análisis detallado**, que muestra los puntos asignados a cada función individual, y
- un **análisis indicativo**, basado en el número de ILF y EIF detectados.

De esta forma, se obtiene la **estimación final en puntos de función**, que puede ser utilizada tanto para la planificación inicial del proyecto como para comparaciones posteriores entre iteraciones o versiones del sistema.

4.3 Prompt

El **diseño** del prompt fue un aspecto clave en el proceso. Se especificó explícitamente el rol que debía adoptar el modelo, las reglas de categorización de cada tipo de función FPA, así como ejemplos positivos y negativos para guiar la clasificación. Además, se incluyó un formato de respuesta esperado en JSON con un ejemplo ilustrativo, y se definieron restricciones claras respecto de qué información no debía ser generada. Finalmente, el conjunto completo de user stories se adjuntó como entrada.

En el diseño del prompt se apoyó en técnicas de **prompt engineering**. En lugar de utilizar un enfoque **zero-shot** (donde el modelo recibe únicamente la instrucción general sin ejemplos), se optó por un esquema **few-shot prompting**, en el cual se incluyen ejemplos de entradas y salidas esperadas para guiar la clasificación.

4.4 Formato de la respuesta del modelo y de la API

El **modelo de IA preentrenado** devuelve una lista de funciones elementales identificadas en las user stories, con su tipo y complejidad estimada:

```
[
  { "funcion": "Cliente", "tipo":
    "ILF", "complejidad": "Alta" },
  { "funcion": "Factura", "tipo":
    "ILF", "complejidad": "Media" },
  { "funcion": "Reporte", "tipo":
    "EO", "complejidad": "Alta" }
]
```

A partir de esta información, el Web Service genera la **respuesta final de la API**, agregando los cálculos de puntos de función detallado e indicativo:

```
{
  "funciones": [
    { "funcion": "Cliente", "tipo":
      "ILF", "complejidad": "Alta" },
    { "funcion": "Producto", "tipo":
      "ILF", "complejidad": "Media" }
  ],
  "calculado_detalhado": 150,
  "calculado_indicativo": 150
}
```

- **funciones:** lista de funciones elementales identificadas y clasificadas.
- **calculado_detalhado:** suma de los puntos de función considerando la complejidad asignada a cada función según NESMA.
- **calculado_indicativo:** estimación preliminar basada en un cálculo simplificado para planificación rápida.

Este flujo permite separar claramente la **etapa de predicción del modelo** y la **etapa de cálculo y entrega final al usuario**, facilitando la integración con otros sistemas y la interpretación automática de los resultados.

4.5 Ejemplo

Bajo el contexto de un **sistema de reservas de hotel**, el Web Service procesa automáticamente un conjunto de user stories provenientes del backlog del proyecto.

A modo de ejemplo, se analizan las siguientes tres user stories:

1. “Como huésped, quiero buscar habitaciones disponibles según fecha y cantidad de personas, para poder realizar una reserva.”
2. “Como huésped, quiero ver las fotos y descripciones de habitaciones, para poder elegir la más adecuada.”
3. “Como huésped, quiero confirmar mi reserva y recibir una factura electrónica, para contar con un comprobante de pago.”

Una vez recibidas, estas historias se combinan con el prompt previamente diseñado y se envían al modelo de **Mistral**.

El modelo clasifica cada historia según las categorías establecidas por **NESMA** (EI, EO, EQ, ILF y EIF) y asigna un nivel de complejidad (baja, media o alta), devolviendo el siguiente resultado en formato JSON:

```
{ "response": [  
  { "funcion": "Buscar habitaciones  
disponibles según fecha y cantidad de  
personas", "tipo": "EQ", "complejidad":  
"media"}, { "funcion": "Ver fotos y  
descripciones de habitaciones",  
"tipo": "EO", "complejidad": "baja" },  
{ "funcion": "Confirmar reserva y  
recibir factura electrónica", "tipo":  
"EO", "complejidad": "alta" }  
]}
```

El **Web Service** recibe esta respuesta y realiza el cálculo correspondiente en base

a la tabla de complejidad definida por NESMA.

Para este conjunto de tres historias, el sistema obtuvo un **resultado detallado de 15 puntos de función**, mientras que el **análisis indicativo** arrojó un valor de **0 puntos**, debido a que no se identificaron funciones ILF o EIF en el conjunto procesado.

Ejemplo de salida de 3 user stories

```
{ "funciones": [  
  { "funcion": "Buscar habitaciones  
disponibles según fecha y cantidad de  
personas", "tipo": "EQ",  
"complejidad": "media" },  
  { "funcion": "Ver fotos y  
descripciones de las habitaciones",  
"tipo": "EO", "complejidad": "baja" },  
  { "funcion": "Confirmar reserva y  
recibir comprobante de pago", "tipo":  
"EO", "complejidad": "alta" } ],  
  "calculado_detallado_PF": 15,  
  "calculado_indicativo": 0 }
```

Al ampliar el análisis a un grupo de **15** user stories dentro del mismo contexto funcional (reserva, registro, pagos y reportes), el sistema estimó un resultado detallado de **75 puntos** de función y un indicativo de **70 puntos** de función.

Estos valores pueden ser utilizados como base objetiva para planificar el esfuerzo de desarrollo, estimar costos y comparar la productividad entre iteraciones o proyectos similares.

5. Limitaciones y alternativas técnicas en el uso de IA

Durante el inicio de la implementación, se decidió trabajar con **modelos de lenguaje preentrenados** para asistir en la clasificación y análisis de user stories. Sin embargo, este enfoque presentó diversos

desafíos relacionados con la disponibilidad, el costo y la infraestructura requerida.

En primer lugar, las plataformas comerciales más consolidadas, como **OpenAI**⁸ o **Gemini** [4], requieren un esquema de pago para su utilización. Para integrar dichos servicios en proyectos con **Spring AI**⁹ o **LangChain4j**¹⁰, es necesario disponer de claves de acceso y un presupuesto destinado al consumo de la API.

Como alternativa gratuita, se evaluó el uso de **Ollama**¹¹, una herramienta que permite descargar y ejecutar modelos localmente. Si bien ofrece la ventaja de prescindir de un servicio en la nube, su rendimiento depende de la capacidad del equipo de cómputo: el consumo de **RAM** y **CPU** puede ser significativo, y en equipos limitados los tiempos de respuesta pueden resultar elevados. La conexión desde la aplicación se realiza mediante la propia librería de Ollama, por lo que la integración es sencilla, aunque condicionada por la infraestructura disponible.

Otra opción explorada fue **Hugging Face**[5], plataforma que ofrece un amplio repositorio de modelos preentrenados de distintos tipos. A través de un **token de acceso** y la **API de inferencia**[6], es posible integrarlos directamente en el flujo de análisis. Sin embargo, no todos los modelos publicados son accesibles mediante este método, lo que restringe el abanico de alternativas disponibles.

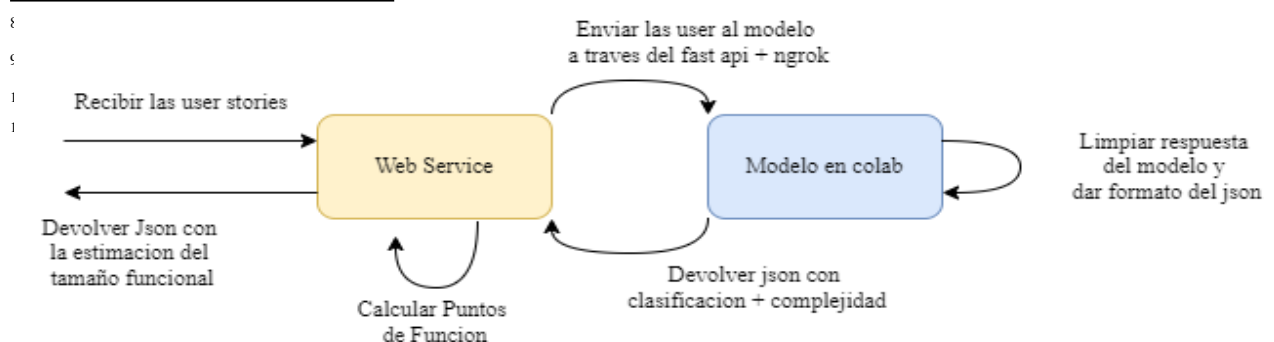
Finalmente, se implementó una estrategia complementaria utilizando **Google Colab** para ejecutar el modelo **Mistral**, disponible en **Hugging Face**, al cual se accede mediante un **token de acceso**. Para este fin se levantó un pequeño servidor web con **FastAPI**, encargado de recibir los prompts enviados desde la aplicación desarrollada en Spring Boot, procesar las user stories a través del modelo de lenguaje y devolver la clasificación correspondiente en formato JSON.

Con el objetivo de permitir el acceso externo, el servicio se expuso mediante **Ngrok**, que genera una **URL pública temporal** redirigiendo las peticiones hacia el servidor alojado en Colab. Esto posibilitó realizar pruebas de integración de forma gratuita y sin depender de la capacidad del hardware local, aprovechando los recursos de cómputo en la nube de Google.

Es importante señalar que, para utilizar Ngrok, es necesario **crear una cuenta** en la plataforma y generar un **token de autenticación**, el cual debe configurarse en el entorno de ejecución de Colab para establecer el túnel seguro y habilitar el acceso externo a la API.

De este modo, la **API REST del Web Service** puede consumir el modelo como si se tratara de cualquier otro servicio externo, manteniendo la independencia entre la lógica de negocio y el procesamiento basado en IA.

5.1 Imagen del flujo implementado



5.2 Ventajas:

1. Permite **ejecutar modelos de gran tamaño** sin depender de la capacidad de la máquina local, lo que es especialmente útil para modelos que requieren alta capacidad de memoria RAM o GPU.
2. No se incurre en costos de suscripción a servicios comerciales, manteniendo la prueba de concepto **gratuita**.

5.3 Limitaciones:

1. La configuración inicial es más compleja, ya que implica levantar el servidor en Colab, instalar dependencias y exponerlo mediante Ngrok.
2. La URL generada por Ngrok es **temporal**, por lo que la sesión de Colab debe estar activa mientras se realizan las peticiones. Esto limita la estabilidad y confiabilidad para entornos de producción.
3. El tiempo de respuesta puede ser mayor en comparación con servicios comerciales optimizados, debido a la latencia de la red y la capacidad compartida del entorno de Colab.

6. Conclusión

Se evaluaron distintas alternativas técnicas para integrar modelos de lenguaje preentrenados, incluyendo plataformas comerciales (OpenAI, Gemini), ejecución local mediante Ollama, Hugging Face y

ejecución remota gratuita a través de **Google Colab + Ngrok**.

Cada enfoque presenta ventajas y limitaciones, destacando que la elección depende del balance entre costo, capacidad de infraestructura y estabilidad requerida.

La solución implementada demuestra la viabilidad de integrar modelos de IA en el proceso de estimación funcional, reduciendo significativamente el tiempo requerido y minimizando la subjetividad del análisis manual.

7. Trabajos futuros

Como línea de trabajo futura, se plantea la posibilidad de **entrenar un modelo propio** ajustado específicamente a proyectos ágiles, lo que permitiría mejorar la precisión en la clasificación de user stories y la estimación de puntos de función. Además, se buscará **optimizar la eficiencia del flujo de procesamiento**, reduciendo tiempos de respuesta y aprovechando mejor los recursos disponibles.

Otro objetivo importante es explorar la integración directa en entornos de producción mediante servicios de IA confiables, eliminando la dependencia de soluciones temporales o de pruebas como Google Colab + Ngrok.

Asimismo, se contempla la integración con herramientas de gestión como **Jira**, de modo que las **user stories** puedan ser recibidas automáticamente desde el backlog del proyecto, procesadas por el modelo de IA y analizadas en términos de Puntos de Función sin necesidad de intervención manual.

Estas mejoras abrirán oportunidades para la **estandarización y automatización de métricas de software**, facilitando la planificación, seguimiento y gestión de proyectos en entornos ágiles y complejos.

Agradecimientos

Expreso mi agradecimiento a los **docentes investigadores y responsables del programa de becarios**, por su constante guía, acompañamiento y dedicación durante el desarrollo de este proyecto.

Su orientación y compromiso fueron fundamentales para afianzar los conocimientos técnicos y metodológicos aplicados en esta investigación.

Asimismo, agradezco profundamente a mis compañeros de equipo, por el apoyo, la colaboración y las valiosas discusiones que enriquecieron cada etapa del trabajo. Su disposición y compañerismo contribuyeron significativamente al logro de los objetivos propuestos.

Referencias

- [1] Nesma FPA standard - Nesma. (2024, 17 febrero). Nesma.
<https://nesma.org/publications/documents/nesma-standard/>
- [2] Hernández, N., Castillo, J. J., Cardenas, M. E., Albarracín, G., & Victoria, J. (2024). Automatización de métodos de estimación basados en puntos de función mediante el análisis de diagramas UML. In *XXVI Workshop de Investigadores en Ciencias de la Computación (WICC)(Puerto Madryn, 18 y 19 de abril de 2024)*.
- [3] Hasan, A. (2024, 18 noviembre). *Using Hugging Face Models in Google Colab: A Beginner's Guide*. DEV Community.
https://dev.to/ajmal_hasan/using-hugging-face-models-in-google-colab-a-beginners-351l
- [4] Caballar, R., & Stryker, C. (2025, 3 junio). Google Gemini. *IBM*.
<https://www.ibm.com/think/topics/google-gemini#:~:text=Gemini%20is%20Google's%20large%20language,software%20code%2C%20text%20and%20video>
- [5] Castaño, J., Martínez-Fernández, S., Franch, X., & Bogner, J. (2024, April). Analyzing the evolution and maintenance of ml models on hugging face. In *Proceedings of the 21st International Conference on Mining Software Repositories* (pp. 607-618).

- [6] Hugging Face Inc., "Hugging Face Hub Documentation,"
<https://huggingface.co/docs/hub/index>, 2023.